

Hausübung 3 - REST, Datenbanken, JDBC

Aufgabe 1

Datenbanken - Normalisierung

Die folgende Tabelle ist gegeben:

album_ID	Album	Titelliste	Gründungsjahr
3718	Blink-182 - One More Time	{1. When We Were Young, 2. Turpentine, 3. Terrified}	1992
5673	Linkin Park - Meteora	{1. Hit the Floor, 2. Easier to Run, 3. Breaking the Habit}	1996

Abbildung 1: Keine Normalform

1. Bringen Sie die gegebene Tabelle 1 in die erste Normalform (**1NF**).
2. Bringen Sie die gegebene Tabelle 1 in die zweite Normalform (**2NF**).
3. Bringen Sie die gegebene Tabelle 1 in die dritte Normalform (**3NF**).

Diese Aufgabe ist in einem geeigneten Format abzugeben. Vorzugsweise eignet sich eine PDF- oder Markdown-Datei.

Aufgabe 2

REST

In der Aufgabe geht es um User einer Webseite, die ihre erstellten Songs abspeichern können. Es können Songs angelegt und für unterschiedliche User abgespeichert werden. Wenn ein User einen Song abspeichert, wird dies in der Datenbank für den User eingetragen. Ein User besteht aus einem Usernamen, einer ID, einer Email und einer Liste von erstellten Songs. Ein Song besteht aus einer SongID, einem Namen, einem Genre und einem Erstelldatum.

1. Für ihre erste Aufgabe sollen Sie konzeptionell Routen erstellen (Nicht implementieren, sondern nur theoretisch in einem Text-Dokument verfassen). Die Funktion der Routen wird für die User und Song Entität unten beschrieben. Bitte geben Sie zu den Routen eine beispielhafte Ausgabe, sowie das passende HTTP-Verb und den Status-Code bei erfolgreicher Anfrage und Antwort an. Geben Sie auch für jede Route mindestens zwei Fehler an und welche Status-Codes dabei am sinnvollsten sind. Die Routen sollen für die folgenden Beschreibungen erstellt werden:
 - (a) User:
 - i. Eine Route, um einen neuen User anzulegen
 - ii. Eine Route, um alle angelegten User zu erhalten
 - iii. Eine Route, um einen bestimmten User zu erhalten
 - iv. Eine Route, um einen User zu löschen
 - (b) Song:
 - i. Eine Route, um einen Song anlegen zu können
 - ii. Eine Route, um einen erstellten Song einem Nutzer zuordnen zu können
 - iii. Eine Route, um sich alle Songs eines Nutzers ausgeben zu lassen
 - iv. Eine Route, um einen Song von einem Nutzer zu entfernen
 - v. Eine Route, um einen Song zu löschen

Diese Aufgabe ist in einem geeigneten Format abzugeben. Vorzugsweise eignet sich eine PDF- oder Markdown-Datei.

2. Nachdem Sie nun konzeptionell ihre Routen definiert haben, sollen Sie diese praktisch umsetzen. Sie sollen aufbauend auf der vorherigen Aufgabe nun einen REST-Service mit Vert.x implementieren, welcher die Entitäten in einer Datenbank (MariaDB) speichert und über HTTP-Routen bereitstellt.

- (a) Legen Sie ein neues Vert.x Projekt an mit dem Vert.x Starter (<https://start.vertx.io/>).
 - i. Als Group ID nutzen Sie bitte wieder *de.thm.mni.pi2*
 - ii. Als Artifact ID nutzen Sie bitte *musicService*
 - iii. Fügen Sie die Vert.x Web Dependency hinzu.
 - (b) In dem erstellten Projekt öffnen Sie die **pom.xml** und fügen Sie die JDBC-Dependency, wie in der Vorlesung gezeigt, hinzu.
 - (c) Beginnen Sie nun mit der Implementierung der (mindestens) neun Routen aus der vorherigen Aufgabenbeschreibung.
 - (d) **Hinweis (1):** Achten Sie in ihrer Implementierung darauf, geeignet auf ungültige Eingaben bei Anfragen zu reagieren (Bsp: Kann man einen Song mit leerem Namen erstellen?). Verwenden Sie passende Status-Codes, um Fehler von Nutzern zu signalisieren, aber auch Nutzer auf mögliche Fehler innerhalb der Anwendung aufmerksam zu machen (Bsp: Status-Code für einen Datenbankfehler?)
 - (e) **Hinweis (2):** Achten Sie auch auf die richtige Vorgehensweise beim Löschen von Inhalten. Was passiert wenn man einen Song löscht, welcher aber noch einem Nutzer zugeordnet ist oder anders herum? Überlegen Sie sich eine plausible Umsetzung für solche Fälle.
 - (f) **Hinweis (3):** Um einen Parameter in einer URL (Bsp. <http://localhost:8888/books/42>) zu behandeln, verwenden Sie Routenparameter:
`router.get("/books/:id").handler(this::handleGetBookById);` beispielsweise legt die Route an. In der Methode `handleGetBookById` können Sie dann mit `routingContext.pathParam("id")` auf den in der URL übergebenen Wert zugreifen.
3. Erstellen Sie die Tabellen mit einem .sql Skript, das Sie mit dem **source**-Befehl einlesen können.
 4. Testen Sie ihre Routen, in dem Sie eine .http Datei anlegen und dort Beispieldaten verwenden, um valide Anfragen an ihren Service zu versenden.

Die Abgabe erfolgt als ein ZIP-File mit dem Namen "HA3-Gruppennummer.zip". Der Ordner-Aufbau innerhalb des zip-Files sieht wie folgt aus:

```
HA3-Gruppennummer.zip
├─ musicService (Dies ist der durch Vertx.io generierte Projekt-Ordner für den Server)
├─ musicService.http
├─ ha3.sql (Dies ist das Skript, um die Tabellen anzulegen)
├─ Aufgabe1.pdf oder Aufgabe1.md
├─ Aufgabe2.pdf oder Aufgabe2.md
```